

Reviewer Pack — Minimal Order of Symmetric Tensors and Frege Proof Length Co...

Entry 6da4aa8000e9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 03:56:55 UTC

Minimal Order of Symmetric Tensors and Frege Proof Length Correlation

Entry ID: 6da4aa8000e9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 03:56:55 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Tensor Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For every boolean function φ with m variables, the minimal order of a symmetric tensor that can represent φ is linearly correlated with its Frege proof length, such that $\text{min_order}(\text{Tensor}(\varphi)) = \Theta(\text{proof_length}(\varphi))$.

Rationale (proposer's reasoning):

Symmetric tensors provide a framework for capturing polynomial representations of boolean functions. The minimal order of a symmetric tensor for a function may reveal the complexity required to prove its truth in the Frege system, potentially leading to lower bounds on proof length.

Taxonomy category: symmetric_tensor_frege_length (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e3c9abf71d4acd30

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the 30 random seeds show a Pearson correlation coefficient greater than or equal to 0.5 between $\text{min_order}(\text{Tensor}(\varphi))$ and $\text{proof_length}(\varphi)$, with no seed producing a correlation coefficient below 0.3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetric tensors AND Frege proof complexity - tensor representation AND minimal order IN BOOLEAN MODE - Frege proof length AND correlation WITH symmetric tensor order

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=5.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(m):
        cnf = []
        for _ in range(2**m - 1):
            clause = [random.randint(-m, m) for _ in range(m)]
            if all(lit != 0 for lit in clause):
                cnf.append(clause)
        return cnf

    def tseitin_encoding(cnf):
        literals = set()
        for clause in cnf:
            literals.update(abs(lit) for lit in clause)
        n_vars = max(literals)

        new_cnf = []
        var_counter = n_vars + 1
        for i, clause in enumerate(cnf):
            tseitin_var = -var_counter
            new_clause = [tseitin_var]
            for lit in clause:
                if lit > 0:
                    new_clause.append(-lit)
                else:
                    new_clause.append(lit)
            new_cnf.append(new_clause)
```

```

    for j in range(len(clause)):
        other_clauses = cnf[:i] + cnf[i+1:]
        for other_clause in other_clauses:
            if lit not in other_clause and -lit not in other_clause:
                new_other_clause = [tseitin_var, -other_clauses[j]]
                new_cnf.append(new_other_clause)

    var_counter += 1

    return new_cnf

def min_order(cnf):
    n_vars = max(abs(lit) for lit in cnf)
    if n_vars == 0:
        return 0
    tensor_order = n_vars
    while True:
        found = False
        for clause in cnf:
            if any(abs(lit) > tensor_order for lit in clause):
                found = True
                break
        if not found:
            return tensor_order
        tensor_order += 1

def dpll_solver(cnf):
    def solve(lits, cls):
        if not lits:
            return []
        unit_clause = next((c for c in lits if len(c) == 1), None)
        if unit_clause:
            lit = unit_clause[0]
            new_lits_true = [l for l in lits if l != lit and -l not in lits]
            new_lits_false = [l for l in lits if l != -lit and l not in lits]
            return solve(new_lits_true, cls) or solve(new_lits_false, cls)
        pure_literal = next((l for l in range(1, max(abs(lit) for lit in lits) + 1) if (l in lits and -l not in lits)), None)
        if pure_literal:
            new_lits_true = [l for l in lits if l != pure_literal]
            new_lits_false = [l for l in lits if l != -pure_literal]
            return solve(new_lits_true, cls) or solve(new_lits_false, cls)
        branching_lit = random.choice(lits[0])
        new_lits_true = [l for l in lits if l != branching_lit and -l not in lits]
        new_lits_false = [l for l in lits if l != -branching_lit and l not in lits]
        return solve(new_lits_true, cls) or solve(new_lits_false, cls)

    return solve(cnf, cnf)

def proof_length(cnf):
    return len(dpll_solver(cnf))

m = random.randint(5, 30)
cnf = generate_cnf(m)
tseitin_cnf = tseitin_encoding(cnf)
min_tensor_order = min_order(tseitin_cnf)
proof_len = proof_length(cnf)

```

```

return {
    "metric_name": "min_order",
    "metric_value": min_tensor_order,
    "instances_tested": 1,
    "n_max": m,
    "conjecture_holds": min_tensor_order >= 0.5 * proof_len and min_tensor_order <= 2 * proof_len,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alccc173.py", line 119, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alccc173.py", line 100, in run_trial
    tseitin_cnf = tseitin_encoding(cnf)
                  ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alccc173.py", line 51, in tseitin_encoding
    new_other_clause = [tseitin_var, -other_clauses[j]]
                        ^^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended function of calculating the correlation coefficients for the conjecture. | next: Re-run the test with proper error handling to ensure that it completes execution and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	30053
2	preregistration	ollama_remote	glm4:latest	0	0	9121
3	novelty	ollama_remote	glm4:latest	0	0	9859
4	novelty	ollama_remote	glm4:latest	0	0	15310
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16111
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	176030
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	142308
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	194569
9	judge	ollama_remote	glm4:latest	0	0	20211

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 613574 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/6da4aa8000e9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6da4aa8000e9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6da4aa8000e9.tar.gz (if generated)